Matrix-Based Image Transformation Tool

A comprehensive, interactive web-based application that uses **linear algebra (matrices)** to manipulate images by transforming pixel positions in real time. Built with pure HTML, CSS, and JavaScript — no external dependencies required.

Table of Contents

- Overview
- Features
- How It Works
  - Mathematical Foundation
  - Transformation Types
- Getting Started
  - Prerequisites
  - Installation
  - Usage
- Transformation Reference
  - Translation
  - Rotation
  - Scaling
  - Shearing / Skewing
  - Reflection
  - Perspective / Projective
- Architecture
  - Project Structure
  - Core Modules
- UI Controls
- Keyboard Shortcuts
- Performance Considerations
- Browser Compatibility
- Comparison with Industry Tools
- Roadmap
- Contributing
- License

Overview

Images are fundamentally 2D arrays (matrices) of pixel data. Every geometric change to an image — rotation, scaling, skewing, reflection, or perspective distortion — can be expressed as a **matrix multiplication** applied to pixel coordinates.

This tool provides an **interactive, visual playground** for exploring and applying these matrix-based transformations. Users can:

- Upload any image and see transformations applied in real time.
- Edit the 3×3 transformation matrix directly.
- Use preset buttons for common operations.
- Compose (chain) multiple transformations together.
- Export the transformed result.

Features

Feature	Description
Image Upload	Load any image (PNG, JPEG, BMP, WebP, SVG) via drag-and-drop or file picker.
Live 3×3 Matrix Editor	Directly edit the nine cells of the transformation matrix and see results instantly.
Preset Transformations	One-click buttons for rotation, scaling, shearing, translation, reflection, and perspective.
Transformation Composition	Chain multiple transformations by multiplying matrices together in sequence.
Transformation History	Undo/redo stack that tracks every matrix change.
Real-Time Canvas Preview	Side-by-side original and transformed image rendered on HTML5 Canvas.
Interpolation Modes	Nearest-neighbor and bilinear interpolation for controlling output quality.
Export	Download the transformed image as PNG or JPEG.
Responsive UI	Works on desktop and tablet screens.
Zero Dependencies	Pure HTML + CSS + JavaScript — no frameworks, no build step.

How It Works

Mathematical Foundation

Every pixel in a 2D image has a coordinate (x, y). To apply a geometric transformation, we represent coordinates in **homogeneous form** as a 3×1 column

vector:

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

A transformation is a **3×3 matrix M** that maps the original coordinate to a new coordinate:

$$\mathbf{p}' = \mathbf{M} \cdot \mathbf{p} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

The resulting coordinate (x', y') is obtained by dividing by the homogeneous component:

$$x' = \frac{a \cdot x + b \cdot y + c}{g \cdot x + h \cdot y + i}, \quad y' = \frac{d \cdot x + e \cdot y + f}{g \cdot x + h \cdot y + i}$$

For **affine** transformations (translation, rotation, scaling, shearing, reflection), the bottom row is always $[0, 0, 1]$. For **perspective** (projective) transformations, g and h can be non-zero.

Transformation Types

Type	Degrees of Freedom	Bottom Row
Affine	6 (a, b, c, d, e, f)	$[0, 0, 1]$
Perspective	8 (a–h)	$[g, h, 1]$

Composition of transformations is achieved by multiplying their matrices. Applying transform **A** followed by **B** is equivalent to a single transform $\mathbf{C} = \mathbf{B} \cdot \mathbf{A}$.

Getting Started

Prerequisites

- A modern web browser (Chrome, Firefox, Edge, Safari).
- No server, build tools, or package manager required.

Installation

```
# Clone or download the repository
git clone https://github.com/your-username/matrix-image-transform.git
cd matrix-image-transform

# Open the application
# Simply open index.html in your browser:
start index.html        # Windows
open index.html         # macOS
xdg-open index.html     # Linux
```

No `npm install`. No bundler. Just open the file.

Usage

1. **Load an image** — Click the upload area or drag-and-drop an image file.
2. **Edit the matrix** — Modify the 3×3 matrix cells in the control panel. Changes apply in real time.
3. **Use presets** — Click a preset button (e.g., "Rotate 45°", "Scale 2×", "Shear X") to populate the matrix.
4. **Compose transforms** — Click **Apply & Compose** to multiply the current matrix onto the accumulated transformation, then set a new one.
5. **Reset** — Click **Reset** to return to the identity matrix.
6. **Export** — Click **Download** to save the transformed image.

Transformation Reference

Translation

Shifts the image by (t_x, t_y) pixels.

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Parameter	Effect
$t_x > 0$	Shift right
$t_x < 0$	Shift left
$t_y > 0$	Shift down
$t_y < 0$	Shift up

Rotation

Rotates the image by angle θ (counter-clockwise) around the origin.

$$\mathbf{R} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

To rotate around the **center** of the image (width w, height h), compose with translation:

$$\mathbf{R}_{center} = \mathbf{T}(w/2, h/2) \cdot \mathbf{R}(\theta) \cdot \mathbf{T}(-w/2, -h/2)$$

Scaling

Resizes the image by factors (s_x, s_y).

$$\mathbf{S} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Value	Effect
$s > 1$	Enlarge
$0 < s < 1$	Shrink
$s < 0$	Reflect and scale

Shearing / Skewing

Distorts the image along one axis.

Shear along X:

$$\mathbf{Sh}_x = \begin{bmatrix} 1 & k_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Shear along Y:

$$\mathbf{Sh}_y = \begin{bmatrix} 1 & 0 & 0 \\ k_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

k_x tilts horizontally; k_y tilts vertically.

Reflection

Mirrors the image across an axis.

Reflect across Y-axis (horizontal flip):

$$\mathbf{F}_y = \begin{bmatrix} -1 & 0 & w \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Reflect across X-axis (vertical flip):

$$\mathbf{F}_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & h \\ 0 & 0 & 1 \end{bmatrix}$$

(Translation terms w and h keep the image within the visible canvas.)

Perspective / Projective

Maps an image so parallel lines converge, creating a 3D depth effect.

$$\mathbf{P} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix}$$

When $g \neq 0$ or $h \neq 0$, the transformation is **non-affine** — straight lines remain straight, but parallelism is not preserved. This is the same math behind `cv::getPerspectiveTransform` in OpenCV.

Architecture

Project Structure

```
matrix-image-transform/
  index.html          # Main application page
  css/
     style.css       # UI styles, layout, theming
  js/
     matrix.js       # 3×3 matrix math (multiply, invert, determinant)
     transform.js    # Pixel-level image transformation engine
     presets.js       # Preset transformation matrices
     ui.js            # DOM event handlers and UI state
     app.js           # Application entry point and initialization
  assets/
```

```
    sample.png      # Default sample image
README.md           # This file
```

Core Modules

`matrix.js` — Matrix Math Engine Implements 3×3 matrix operations used throughout the tool:

Function	Description
`multiply(A, B)`	Multiplies two 3×3 matrices, returning `C = A × B`.
`invert(M)`	Computes the inverse $\mathbf{M}^{-1}$ using the adjugate method.
`determinant(M)`	Calculates the determinant $\det(\mathbf{M})$.
`identity()`	Returns the 3×3 identity matrix.
`transpose(M)`	Returns $\mathbf{M}^T$.
`applyToPoint(M, x, y)`	Transforms a single point (x, y) through matrix $\mathbf{M}$, returning (x', y') with perspective division.

`transform.js` — Image Transformation Engine Applies the matrix transformation to image pixel data:

1. **Inverse mapping** — For each pixel (x', y') in the output canvas, compute the corresponding source pixel $(x, y) = \mathbf{M}^{-1} \cdot (x', y', 1)$.
2. **Interpolation** — Sample the source image at (x, y) using either nearest-neighbor or bilinear interpolation.
3. **Boundary handling** — Pixels mapping outside the source image are rendered as transparent.

This approach (inverse mapping) avoids holes in the output that forward mapping would produce.

`presets.js` — Preset Transformations Factory functions that generate transformation matrices from user-friendly parameters:

Preset	Parameters
`rotation(degrees)`	Angle in degrees
`scale(sx, sy)`	Scale factors
`translate(tx, ty)`	Pixel offsets
`shearX(k)` / `shearY(k)`	Shear factor
`reflectX()` / `reflectY()`	No parameters
`perspective(g, h)`	Perspective coefficients

`ui.js` — **User Interface Controller** Manages all DOM interactions:

- Matrix grid input handling with validation (numeric-only).
- Preset button bindings.
- Image upload (file input + drag-and-drop).
- Undo/redo history stack.
- Canvas rendering coordination.
- Export functionality.

`app.js` — **Application Entry Point** Initializes the canvas contexts, loads the default sample image, binds all event listeners, and starts the render loop.

UI Controls

Control	Location	Description
Image Upload	Top panel	Drag-and-drop zone or click-to-browse for image files.
Matrix Grid	Left panel	Nine editable input fields arranged as a 3×3 grid.
Preset Buttons	Below matrix grid	Quick-apply buttons for common transforms.
Parameter Sliders	Below presets	Adjust rotation angle, scale factor, shear amount, etc.
Apply & Compose	Action bar	Multiplies the current matrix into the accumulated transform.
Reset	Action bar	Resets to the identity matrix.
Undo / Redo	Action bar	Steps through the transformation history.
Interpolation Toggle	Settings	Switch between nearest-neighbor and bilinear.
Download	Action bar	Exports the transformed image as PNG.
Original Canvas	Right panel (left)	Shows the unmodified uploaded image.
Transformed Canvas	Right panel (right)	Shows the live result of the current matrix.

Keyboard Shortcuts

Shortcut	Action
`Ctrl + Z`	Undo
`Ctrl + Y`	Redo
`Ctrl + S`	Download transformed image
`Ctrl + R`	Reset to identity matrix
`Ctrl + O`	Open image file picker

Performance Considerations

Technique	Purpose
Inverse mapping	Eliminates holes in output; every output pixel is computed exactly once.
Canvas 2D `setTransform()`	For affine transforms, the browser's native GPU-accelerated path is used instead of per-pixel JavaScript.
Per-pixel fallback	Used only for perspective (non-affine) transforms where `setTransform()` cannot be applied.
`ImageData` direct manipulation	Pixel-level transforms read/write the raw RGBA buffer for maximum throughput.
Debounced input	Matrix cell edits are debounced (50 ms) to prevent excessive re-renders during fast typing.
Web Workers (planned)	Heavy per-pixel transforms can be offloaded to a background thread.

Browser Compatibility

Browser	Minimum Version	Notes
Chrome	80+	Full support
Firefox	78+	Full support
Edge	80+	Full support (Chromium-based)

Browser	Minimum Version	Notes
Safari	14+	Full support
IE 11		Not supported (no ES6 modules, no Canvas 2D features)

Comparison with Industry Tools

Tool	Type	Matrix Editing	Real-Time	Free	Custom Transforms
This Tool	Web App	Direct 3×3			
OpenCV	Library	Via code			
NumPy + PIL	Library	Via code			
scikit-image	Library	Via code			
MATLAB	IDE	Via code			
GeoGebra	Desktop/Web	Visual			Limited
Photoshop	Desktop	UI-driven			Limited
PhotoSauce	.NET Library	Via code			

This tool fills the gap between code-only libraries (OpenCV, NumPy) and UI-only editors (Photoshop) by providing a **visual, interactive matrix editor** with full mathematical control.

Roadmap

- ☐ **Core application** — Image upload, matrix editor, canvas rendering, preset transforms.
- ☐ **Bilinear interpolation** — Smooth output for rotations and scaling.
- ☐ **Transformation chaining UI** — Visual pipeline showing composed matrices.
- ☐ **Animation mode** — Animate smooth transitions between identity and target matrix.
- ☐ **Web Worker rendering** — Offload per-pixel perspective transforms to a background thread.
- ☐ **GPU acceleration (WebGL)** — Use shaders for real-time perspective transforms on large images.
- ☐ **Batch processing** — Apply the same matrix to multiple images.
- ☐ **Matrix decomposition display** — Show the SVD or QR decomposition of the current matrix.
- ☐ **Touch/gesture support** — Pinch-to-scale and rotate on mobile devices.
- ☐ **Dark/light theme toggle** — User-selectable color scheme.

Contributing

Contributions are welcome! To contribute:

1. Fork the repository.
2. Create a feature branch: `git checkout -b feature/my-feature`.
3. Commit your changes: `git commit -m "Add my feature"`.
4. Push to the branch: `git push origin feature/my-feature`.
5. Open a Pull Request.

Please ensure: - Code follows the existing style (vanilla JS, no frameworks). - All matrix operations have corresponding unit tests. - UI changes are tested across Chrome, Firefox, and Safari.

License

This project is licensed under the **MIT License**. See LICENSE for details.

Every image transformation is just a matrix multiplication away.